The command completed successfully.

From this output, you can see that the server at \\192.168.1.1 has three separate shared folders available - images, data, and disk. You can then use this information to connect to the file share and map it as a network drive, which we'll discuss how to do in the next section.

CONNECTING TO A FILE SHARE

To connect to a file share, you again use the NET command. To connect, however, you'll use the NET USE command, rather than NET VIEW. If you wanted to connect to the data fileshare on \\192.168.1.1 and map it as your computer's H drive, you would use this command:

NET USE H: \\192.168.1.1\data

This will mount the shared folder "data" at \\192.168.1.1 as your computer's H drive.

Unfortunately, if you reboot your computer, or log out and then log back into your account, this will break your connection to the share, and you'll have to remap the network drive all over again. You can avoid this by using NET USE with the /PERSISTENT switch:

NET USE H: \\192.168.1.1\data /PERSISTENT:YES

This will tell Windows to remap the network drive every time you log into your account. If the network share is available when you log in, Windows will remap the drive for you.

You might also need to connect to the share as a different user. This can be accomplished via the /USER switch with the NET USE command. If you wanted to connect to the \\192.168.1.1\data share using the user account named camalas, you would use the NET USE command with these parameters:

NET USE H: \\192.168.1.1\data /USER:CAMALAS

This will connect you to the network share at \\192.168.1.1\data using camalas's user account.

DISCONNECTING FROM A FILE SHARE

If you no longer need a particular network drive, you might want to disconnect it. Just as with mapping a network drive, you can disconnect a network drive from the Command Prompt via the NET USE command. To disconnect the H drive we mapped in the earlier example, you would use this command:

NET USE H: \\192.168.1.1\data /DELETE

Using the /DELETE switch might seem rather drastic. Fortunately, the /DELETE switch is misnamed. It doesn't delete any of the files or subdirectories contained in the file share. It only disconnects the mapped network drive from your

computer, and you can reconnect to it at your leisure.

CREATE A FILE SHARE

In addition to connecting to file shares from the Command Prompt, you can also create your own file shares using the command line. Windows Vista and Windows 7 are generally called "client systems", since they're designed to interact with larger server systems running a version of Windows Server. However, the client versions of Windows can act as limited server systems. You can created shared folders on your Windows client system, and allow users to connect to them remotely.

As with finding and connecting to network shares, you use the NET command to create file shares on your Windows computer. To create network shares, you use the NET SHARE version of the NET command. Let's say you have a directory named data in the root directory of your C drive you want to share. To make it into a shared folder, use this command:

NET SHARE DATA=C:\DATA

This will set C:\DATA as a shared folder with a share name of "DATA." The new share won't do you very much good unless you assign share permissions to users. You also use the NET SHARE command to assign share permissions to users. For instance, if you wanted to grant the camalas user account the Read permission to the DATA share:

NET SHARE DATA /GRANT:CAMALAS READ

This will assign the camalas user account the Read share permission. To assign the Change permission to the camalas user account, use this command:

NET SHARE DATA /GRANT:CAMALAS CHANGE

And to grant the Full share permission:

NET SHARE DATA /GRANT:CAMALAS FULL

Finally, to view all the file shares on your system, use the NET SHARE command without any arguments:

NET SHARE

This will generate a similar output to using NET VIEW to view the file shares on a remote system, but this listing will show only the file shares on the local system.

***

Chapter 11 - Managing user accounts

To use a Windows computer, you need a user account on that computer. Most home computers generally have one local account with administrative rights, and the user never gives it another thought. Windows computers that are part of a large business network are usually members of an Active Directory domain, with their user accounts stored on one of the domain's controllers. Active Directory is beyond the scope of this book, but in this chapter, we'll show you how to create and manage local accounts from the Command Prompt.

WHAT IS A USER ACCOUNT?

Before we begin, you should have a clear idea of what a user account is, and the difference between local user accounts and Active Directory user accounts.

To put it simply, a "user account" is a set of credentials that allow a user to log into a Windows computer system. "Credentials", in this context, is the term that computer security experts use to describe the pieces of information a user must present in order to access the computer. Most of the time, the credentials consist of two pieces of information - your username, and your password. However, you can have additional credentials, as well - more secure computer systems might require you to present a username, a password, and a smart card, or a USB flash drive with an encryption key stored upon it. Advanced systems can even require biometric credentials - your fingerprints, for instance. The vast majority of home computers are not so secure, and usually require only a username and a password (and some home computers are configured not even to require a password, which is always a bad idea).

There are two kinds of user accounts you will encounter in a Windows system - local user accounts, and Active Directory domain user accounts. Local user accounts are stored on one specific Windows computer, and work only on that particular Windows system. If you have a computer named System1, and a user account called camalas on that computer, then the camalas account will only work on System1. If you tried to use the camalas user account on another computer named System2, it would not work. You could set up an identically named local account on that computer named camalas, but it would not be the same account. Local accounts work fine for individual computers, or for computers on smaller networks, but managing local user accounts on a large network of hundreds or thousands of computers quickly becomes impractical.

This where Active Directory user accounts come in. In Active Directory, individual Windows computers are joined to the Active Directory domain. Each Active Directory domain controller stores a database of user accounts, and a user can log into any domain computer using his Active Directory account. The local accounts still exist on the domain member computers - most organizations leave a local administrator account in case of technical trouble, and perhaps a restricted local account for guest users.

In this chapter we'll focus on managing local accounts from the Command Prompt.

FINDING USER ACCOUNTS

The first step managing local user accounts is to know what accounts you actually have on your system. As with managing file shares, you can accomplish this with the NET command. This time, we'll be using the NET USER variation of the NET command. To find out the names of all the local accounts on your Windows system, issue the NET USER command without any arguments or switches:

NET USER

The output should look something like this:

User accounts for \\SYSTEMNAME

Administrator Guest camalas

The command completed successfully.

In this example, NET USER is listing all the local accounts for the \\SYSTEMANME Windows system. You can see that the computer has three accounts - the built-in Administrator account (which is disabled by default in Windows Vista and Windows 7), the built-in Guest account (also disabled by default), and the camalas account.

DISPLAYING INFORMATION ABOUT A SPECIFIC USER ACCOUNT

NET USER without any options lists the accounts on a system, but provides no other information about them. To get specific information about an individual local account, use the NET USER command with the name of the account. For

instance, to find out specific information about the camalas account, use this command:

NET USER CAMALAS

If the account name has a space in it (as local accounts on a home computer often do), you'll need to enclose the account name in quotation marks:

NET USER "CAINA AMALAS"

Regardless of the account name, the output should look like this:

User name Caina Amalas

Full Name

Comment

User's comment

Country code 000 (System Default)

Account active Yes

Account expires Never

Password last set 4/20/2010 8:00:30 PM

Password expires Never

Password changeable 4/20/2010 8:00:30 PM

Password required No

User may change password Yes

Workstations allowed All

Logon script

User profile

Home directory

Last logon 9/10/2011 12:32:42 PM

Logon hours allowed All

Local Group Memberships *Administrators *Debugger Users

Global Group memberships *None

There's quite a bit of information here, but we'll go over the more important bits.

"User name", obviously, is the actual user name, the name of the account. "Full name" is often blank, but it's sometimes used if the username is a truncated version of the user's full name - a username of "camalas" with a full name of "Caina Amalas", for example. "Account active" indicates whether or not the account is disabled, and "account expires" indicates when the account is going to expire - user accounts can be configured to expire on a certain date (for a temporary employee, for instance).

The section of information dealing with the password is important. It shows when the password was last set, when it expires, and whether or not the user can change his password. It is possible to set a password policy forcing users to change their passwords after a set period of time. You can see the date of the password's expiration in NET USER's output, as well.

"Last logon", as the name indicates, shows the last time the user account logged

into the computer. "Logon hours allowed" displays the hours when the account can log into the computer. This is useful if you want to restrict the times a user can use his account - a work employee can log in only during business hours. Or if you have children, you can restrict the hours they can use the family computer.

Finally, the "Local Group Memberships" field shows the local groups to which the user account belongs. If you want a user to have administrative rights, the account needs to be a member of the Administrators user group.

ADDING A USER ACCOUNT

NET USER can be used to view information about accounts already on your Windows system, but you can also employ it to add additional accounts. The basic format is to use NET USER with the /ADD switch and the account name. So to add a user account named camalas to your system, you would use this command:

NET USER CAMALAS /ADD

This command will add a user account of camalas to your computer. Note that an account name can only have a maximum of twenty characters.

However, you can also use NET USER and /ADD in conjunction with other command switches. You can also set the password associated with the camalas account right away through the use of the /PASSWORDREQ switch. To set camalas's account with a password of Cymr!Yy29 (note that passwords, unlike the rest of Command Prompt, are case senstive):

NET USER CAMALAS Cymr!Yy29 /ADD /PASSWORDREQ:YES

You can also set a full name for the account right away by using the /FULLNAME switch:

NET USER CAMALAS /ADD /FULLNAME:"Caina Amalas"

RESTRICING USER ACCOUNTS

Now that you know how to add user accounts, you may need to restrict the times that users can log into the system. You can do this by using NET USER with the /TIME switch. For example, to set that camalas account so that it can only log into the system on Mondays through Fridays from 6AM to 6PM:

NET USER CAMALAS /TIME:M-F, 06:00-18:00

This will restrict camalas from using the computer from 6 AM until 6 PM on Mondays through Fridays.

DISABLING USER ACCOUNTS

You might find it necessary from time to time to disable a user account. Usually, this happens when an employee is fired unexpectedly, and you need to lock his

account to prevent him from accessing sensitive information. Or, if you are a parent, and your child misbehaves, you can punish him by locking his account on the family computer. Disabling a user account deactivates it without deleting it entirely - the account is still there, but locked, and cannot be used to access the computer. To lock the camalas user account from the command line, use this command:

NET USER CAMALAS /ACTIVE:NO

This will disable the camalas user account. To reactivate it, use the same command, but change the /ACTIVE switch from NO to YES:

NET USER CAMALAS /ACTIVE:YES

This will reactivate the account.

DELETING USER ACCOUNTS

Disabling user accounts is a useful ability, but if you have no further need of an account, there's no reason to keep it on your computer. Furthermore, an unused account represents a security risk - if an attacker were to guess the password, he would have access the account. And if the compromised account is an administrative account, he would have complete access to all of your computer's files and settings.

It is generally best to disable accounts rather than delete them entirely - there's

always a chance you might need the account in the future, after all, and disabling an account is almost as secure as deleting it entirely. However, if you're absolutely certain you have no further need of an account, you can delete it using the NET USER command with the /DELETE switch. To delete the camalas account from our previous examples, you would use this command:

NET USERS CAMALAS /DELETE

This will delete the camalas user account. Note that the account is gone forever - if you need to bring it back, you will have to create a new account with the same name.

***

Chapter 12 - Remote access

For several years, Mac OS X and Linux systems have had a significant advantage over Windows systems. Using a technology called SSH (SSH stands for Secure Shell), Mac OS X and Linux system administrators could remotely log into their systems via the command line. Once logged in, they could control their remote systems through the command line, operating the computer as if they were sitting in front of it.

Windows, of course, has had remote access technology for years, via the Remote Desktop or Terminal Services software. Using Remote Desktop, you could remotely log into a server computer, seeing an entire graphical desktop on your computer. This is useful, but Remote Desktop has a few disadvantages over the text-based SSH – it's often quite slower, and on a slow connection with a lot of latency, Remote Desktop can become sluggish. And as with all GUIs, Remote Desktop is less efficient than the command line. To perform a task on a remote computer, you would have to fire up the Remote Desktop client, log in, and then execute the task via the mouse. Using SSH, it's far quicker to execute tasks on a remote Mac OS X or Linux machine – you merely log in using the SSH shell, type the command, and the task is finished. Windows lacked a software feature that could perform a similar remote management function.

All this changed with the introduction of WinRM – Windows Remote Management.

WHAT IS WINRM?

WinRM stands for Windows Remote Management. It is a Microsoft

implementation of the SOAP protocol that lets you manage and control remote Windows machines over a network connection. The "SOAP" in the SOAP protocol stands for Simple Object Access Protocol, a set of rules for exchanging information over an HTTP web connection. One weakness of SSH is that it runs over specific TCP/IP ports (traditionally port 22, though most SSH server software lets you change the port), and that specific ports needs to be unblocked to allow SSH connections. SOAP runs over HTTP, and many firewalls allow HTTP traffic. Therefore, it's possible to run WinRM without opening up any additional firewall ports.

WinRM runs on Windows XP, Windows Vista, Windows 7, Windows Server 2003, Windows Server 2003 R2, Windows Server 2008, and Windows Server 2008 R2. (Though you'll need to install some patches from Microsoft in order to use WinRM on Windows XP.) WinRM has some substantial limitations, but it is quite useful, and hopefully Microsoft will improve it with future versions of Windows.

In the next section we'll walk you through doing a basic setup of WinRM.

ENABLING WINRM

Performing a basic setup of WinRM is extremely easy, and takes only one command. First, launch a Command Prompt as an administrator (the setup will not work if you run the Command Prompt as a normal user). After the Command Prompt window launches, type this command:

WINRM QUICKCONFIG

After you type this command, it will ask if you want to proceed. Hit Y to proceed, and WinRM will activate itself (though you may need to reboot before it functions). What exactly does this command do?

First, it starts the WinRM service, and sets it to delayed automatic start (this means the service will start automatically when your computer boots, but only after other, more important services have first launched). Second, it configures Windows to accept commands from remote computers, even commands that require administrative rights. Third, it tells the WinRM listener to accept requests from the HTTP port. Finally, it opens a firewall exception in the local Windows firewall, so that the HTTP traffic can actually reach the WinRM listener.

Now that WinRM is enabled, you can use it to launch commands from a remote computer.

USING WINRS

To actually send commands to a computer running WinRM, you need to use the WINRS command. WINRS is short for “Windows Remote Shell”, and allows you to send command from your local Command Prompt to a remote computer running WinRM. Note that you must have an administrative account (or a domain account with administrative rights) on the target computer for WINRS to execute your command.

In this example, we’ll assume you want to send remote commands to a computer named System1, and that the computer is a member of the Fake.com domain. The DIR command is a good one to test with WINRS. This command will produce a listing of the root directory of the C drive on System1:

WINRS –R:SYSTEM1.FAKE.COM DIR C:

Let’s go through the options in this command step-by-step.

WINRS, of course, launches the WINRS shell for sending commands to a remote system. The -R switch tells WINRS which computer should receive the command. The -R switch is followed by the colon (:) character, and immediately after the colon is the name of the computer that should receive the command. For the basic set up of WinRM, you'll need to use the FQDN (fully qualified domain name) of the remote computer to receive the command. Finally, after the FQDN comes the command to send to the remote computer.

You can launch any command you want via WINRS, with one important exception. The command cannot require any user input. Basically, if a command needs user input - like if it asks you to press Y to continue or N to cancel - it will not work. This does limit what you can do with WINRS, but with careful planning, you can avoid the limitations. (Like by using /Y switch with the XCOPY and COPY commands to keep them from asking for confirmation when overwriting a file.) However, some commands, like DISKPART, simply cannot be used via WINRS.

USING WINRS AS A DIFFERENT USER

Sometimes, you may need to run a remote command as a different user than your current account. WINRS also gives you this ability. To run a command as a different user, use the -U switch with WINRS. For instance, if you wanted to view the root directory of the C drive on System1 using the camalas account, you would use the WINRS command with this syntax:

WINRS -R:SYSTEM1.FAKE.COM -U:CAMALAS DIR C:

WINRS will ask you for camalas's password. Assuming you enter the password (and assuming that camalas has administrative rights on the target system), the command will list the files in the root directory of System1's C drive.

***

Chapter 13 - Batch files

As you have probably realized by now, while using the Command Prompt is in many circumstances more efficient than using the GUI, it nonetheless involves a great deal of repetitive typing. However, there are way to automate commands, and the easiest way to automate commands from the Command Prompt is to use a batch file. With a batch file, you can launch a dozen commands (or more) in sequence simply by typing the name of a single file from the command line.

In this chapter we'll discuss how to use a simple batch file.

WHAT IS A BATCH FILE?

To put it simply, a batch file is a text file containing a sequence of commands, one in each line. When you launch the batch file from the command line, Command Prompt reads the file, executing each of the commands in sequence. (That means Command Prompt first executes the command on the first line, then the second, then the third, and so on until it reaches the end of the file.)

Using a batch file, you can automate tedious jobs, or tasks that require typing several command in sequence. For instance, let's say you have a daily task on your Windows computer that requires you to enter twelve commands in sequence to complete it. You could type out each command one by one, with the attending risk of making a typing error. Or you could put the commands into a batch file named JOB.BAT, and run it from the command line. Command Prompt will execute each of the commands in sequence, and you needn't worry about any typographical errors.